KLAi

# Python for Data Analysis: instructor guide

*Session-by-session plans, answers and checks for all 24 classes.*

Holiday Data and AI Camp | The National ICT Innovation Hub, Nakawa

Jotham Martin Wambi, main tutor

# Contents

Private to the instructor. Each guide has the 180-minute plan, the answers, the code and the common errors for one session. Use the bookmarks panel to jump to a session.


| | |
|---|---|
| **DA01** | Open a notebook and read your first table |
| **DA02** | Values, names and types |
| **DA03** | Arithmetic, revenue and reading errors |
| **DA04** | Conditions and decisions |
| **DA05** | Lists, dictionaries and loops |
| **DA06** | Functions and checks that fail loudly |
| **DA07** | A table in Python: DataFrame, columns and reading a CSV |
| **DA08** | Select, filter and calculate a revenue column |
| **DA09** | Sort, group, mean and median |
| **DA10** | Cleaning without hiding evidence |
| **DA11** | Charts that say what the numbers say |
| **DA12** | Show your first chart and explain what it says |
| **DA13** | NumPy: the numbers underneath pandas |
| **DA14** | Exploring distributions with seaborn |
| **DA15** | Exploring relationships: scatter plots, correlation and cause |
| **DA16** | Cleaning the raw export |
| **DA17** | Merging two tables and the fan-out trap |
| **DA18** | Does your total match the true total? Clean, merge and reconcile |
| **DA19** | Project question and a safe plan |
| **DA20** | Build a reliable analysis |
| **DA21** | Build and check the chart report |
| **DA22** | Report and handover |
| **DA23** | Explain it to a parent |
| **DA24** | Parent showcase and next steps |

# DA01: Open a notebook and read your first table

Private. Keep this file out of the student folder.

**Objective.** By the end: you can open, run, save and reopen a notebook, and read a table of shop sales into it.

**Question of the day.** What does one week of sales in a small shop look like?

> **WHERE THIS IS USED**
> Supermarkets, mobile money agents and clinics all start the day by opening a table of what happened yesterday. Today you open your first one, from a small shop. What would you most want to know about a shop after seeing one week of its sales?

Prerequisite: START_HERE.md.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 20 | welcome, laptop check, folders and the AI policy |
| 40 | Block A |
| 15 | break away from screens |
| 45 | Block B |
| 40 | Make |
| 20 | exit ticket, save and an individual check with each learner |

Where it overruns: Opening the notebook and finding the data file takes longer than the code. Walk every learner through the folder before the first run.

## Block A. Run your first cell

Read AI_USE_POLICY.md with your instructor now. It explains when AI tools are used in this camp and when they are not. Then click the code cell below and press Shift and Enter. Python is a language for giving a computer instructions. The word print tells it to show something. The brackets hold what to show.

Ask for the prediction first: Predict the two lines that will appear under the cell.

```
print("Hello, data")
print(3 + 2)
```

Correct result: The output is Hello, data on the first line and 5 on the second. Text sits between matching quotation marks. A number such as 3 is an amount.

Investigate, with answers:

- Why does the first line keep its quotation marks in the code and lose them in the output? Answer: The quotation marks tell Python that the words are text. They are part of the instruction and not part of the message.

- What would print(3 + 2) show if the 3 were written as "3"? Answer: It would stop with an error, because Python cannot add text to a number.

Modify: Change Hello, data to your nickname and run the cell again. Answer: The first line now shows the nickname.

```
print("Nyanzi")
print(3 + 2)
```

## Block B. Read the shop table

The file camp_shop.csv holds 60 sales from a practice shop. It has to sit in the same folder as this notebook. The three lines below are ready-made. You will learn how each part works in session 7. For now, read what comes out. The word head means the first rows.

Ask for the prediction first: Predict how many rows the table shows and which columns it has.

```
import pandas as pd
shop = pd.read_csv("camp_shop.csv")
print(shop.head())
```

Correct result: The output shows the first five sales as a table. The columns are sale_id, date, item, units and unit_price_ugx. Sale 1 is one Book at 3000 shillings.

Investigate, with answers:

- How many columns does the table have, and what does each one record? Answer: Five. The sale number, the date, the item sold, the units sold and the price of one unit in Uganda shillings.
- Why does the first row show a sale of only 1 unit while the fourth shows 7? Answer: Different customers buy different amounts. The table records each sale as it happened.

Modify: Change head() to head(10) and run it again. Answer: Ten rows appear.

```
ten = shop.head(10)
```

## Make: reference

```
print("Nyanzi")
print(7 + 4)
import pandas as pd
shop = pd.read_csv("camp_shop.csv")
print(shop.head(3))
# The numerical result is 11.
```

Build log entry to look for: DA01.ipynb that reopens with the nickname, 11 and three table rows.

## Misconceptions to listen for

- A notebook is a Word document.
- A text cell can run code.
- Pressing Enter runs a cell. Shift and Enter does.

## Questions to ask, with expected answers

- What is the difference between a text cell and a code cell? Expected: A text cell holds words for people. A code cell holds instructions that Python runs.
- Where does the output appear? Expected: Directly under the cell that produced it.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Welcome, camera and sound check, and a notebook that opens |
| 35 | Block A, Run your first cell: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Read the shop table: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Run your first cell): Change Hello, data to your nickname and run the cell again.
2. Modify, Block B (Read the shop table): Change head() to head(10) and run it again.
3. Make: In a new code cell show your nickname and the result of 7 + 4. Then show the first three rows of the shop table with head(3). Save the notebook as DA01.ipynb, close it and open it again. On your progress map, write one topic you love, such as football, music or farming. It will grow into your project.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: What is the difference between a text cell and a code cell? Expected: A text cell holds words for people. A code cell holds instructions that Python runs.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA02: Values, names and types

Private. Keep this file out of the student folder.

**Objective.** By the end: you can store numbers and text under names, and say what type each one is.

**Question of the day.** What kind of thing is each part of a sale?

> WHERE THIS IS USED
> A till keeps a product name as text, a price as a number and a date as a date. If the types get mixed up, the totals go wrong. Which one is a phone number: text or a number?

Prerequisite: DA01 Open a notebook and read your first table.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

## Block A. Names point to values

A shop sells 3 Juice bottles at 2000 shillings each. The equals sign stores a value under a name. The right side is worked out first.

Ask for the prediction first: Predict the three lines the cell will print.

```
price = 2000
units = 3
revenue = price * units
print(revenue)
print(type(price))
print(type("2000"))
```

Correct result: The cell prints 6000, then <class 'int'>, then <class 'str'>. price and units are variables. 2000 is an integer. The quoted "2000" is a string, which is text. The equals sign stores the value on its right under the name on its left.

Investigate, with answers:

- Why is type("2000") a string? Answer: Quotation marks make it text. Python cannot multiply it like an amount of money.
- What does the equals sign do in revenue = price * units? Answer: It works out the right side first and then stores the answer under the name revenue.

Modify: Change units to 5 and run the whole cell again. Answer: revenue becomes 10000.

```
units = 5
revenue = price * units
```

## Block B. Decimals and converting text

Prices can have decimals. Text that looks like a number has to be converted before Python can calculate with it. The words float and int convert text into those types.

Ask for the prediction first: Predict what each of the first three prints shows, and whether the last line will work.

```
price = 2500.5
text_price = "2500.5"
print(type(price))
print(type(text_price))
print(price * 2)
print(float(text_price) * 2)
```

Correct result: The cell prints <class 'float'>, <class 'str'>, 5001.0 and 5001.0. The quoted price is text. float() turns it into a number that can be multiplied.

Investigate, with answers:

- Why does the answer 5001.0 keep a decimal point even though 2 is a whole number? Answer: Multiplying a float by 2 gives a float. Python keeps the decimal point.
- What would happen with float("two")? Answer: An error. Only text that looks like a number can be converted.

Modify: Convert the text "2500.5" to a number with float() and multiply it by 4. Answer: 10002.0.

```
total = float("2500.5") * 4
```

## Make: reference

```
price = 2500.5
quantity = 2
print(price * quantity)
print(type("2500.5"))
# 5001.0. With quotation marks 2500.5 is a string, which is text and not an amount.
```

Build log entry to look for: DA02.ipynb with the total 5001.0 and a sentence about the quotation marks.

## Misconceptions to listen for

- The equals sign asks whether two things are equal.
- Quotation marks around a number change nothing.
- Changing units changes revenue automatically.

## Questions to ask, with expected answers

- What does the equals sign do? Expected: It stores the value on the right under the name on the left.

- Why is 5001.0 shown with a decimal point? Expected: Multiplying by a float gives a float.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Names point to values: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Decimals and converting text: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Names point to values): Change units to 5 and run the whole cell again.
2. Modify, Block B (Decimals and converting text): Convert the text "2500.5" to a number with float() and multiply it by 4.
3. Make: Create a decimal price of 2500.5 and a quantity of 2. Print the total. Write one sentence on why "2500.5" with quotation marks behaves differently.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: What does the equals sign do? Expected: It stores the value on the right under the name on the left.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA03: Arithmetic, revenue and reading errors

Private. Keep this file out of the student folder.

**Objective.** By the end: you can calculate revenue for a sale, check it by hand, and read an error message to find the line to fix.

**Question of the day.** How much money did the first three sales bring in?

> **WHERE THIS IS USED**
> Revenue is units times price, added up. A shop owner does it by hand, and a program does it for every sale in a blink. Which error message have you seen on your phone, and what did it mean?

Prerequisite: DA02 Values, names and types.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

Where it overruns: Students enjoy breaking code on purpose. Keep the deliberate errors in separate cells so working code stays clean.

## Block A. Revenue by hand and by Python

Revenue is units times unit price. The first three sales in the shop table are 1 Book at 3000, 2 Pens at 1000 and 4 Juice at 2000. Work the three revenues out on paper first. A line that starts with # is a comment.

Ask for the prediction first: Predict the three revenues and their total, using paper.

```
# revenue = units * price
book = 1 * 3000
pen = 2 * 1000
juice = 4 * 2000
total = book + pen + juice
print(book, pen, juice)
print(total)
print(2 + 3 * 4)
print((2 + 3) * 4)
```

Correct result: The cell prints 3000 2000 8000, then 13000, then 14 and 20. Multiplication happens before addition. Brackets change the order.

Investigate, with answers:

- Why does 2 + 3 * 4 give 14 and not 20? Answer: Python multiplies first, so it works out 3 * 4 = 12 and then adds 2. Brackets are needed to add first.
- Why do we say revenue and not profit? Answer: Revenue is the money received. Profit would need the cost of the stock, which the table does not have.

Modify: Change the Pen to 5 units and find the new total. Answer: 16000.

```
pen = 5 * 1000
total = book + pen + juice
```

## Block B. Reading an error message

An error is feedback about an instruction. It is not a judgement about you. Read the last line first. It names the kind of error. Then find the line number it points to. Type this line in a new cell and run it: print(10 / 0). Then type print(units) in another cell before you have created units.

Ask for the prediction first: Predict the name of each error before you run the lines.

Correct result: The first line stops with ZeroDivisionError: division by zero, because no number answers 10 divided by 0. The second stops with NameError: name 'units' is not defined, because Python has not met a line that creates units. In both cases the last line of the message says what went wrong.

Investigate, with answers:

- Which line of the message do you read first, and why? Answer: The last one. It names the error and says what happened.
- How do you fix a NameError? Answer: Run the cell that creates the name, or correct its spelling and capital letters.

Modify: Fix the NameError by creating units = 4 first and then printing units * 2000. Answer: 8000.

## Make: reference

```
print(float("12.5") * 4)
# 50.0. No number multiplied by zero gives 10, so 10 divided by 0 has no answer.
```

Build log entry to look for: DA03.ipynb with 50.0, the prediction written before the run, and an error log with two errors read correctly.

## Misconceptions to listen for

- An error means the student is bad at this.
- Python reads left to right and ignores bracket rules.
- Revenue and profit are the same.

## Questions to ask, with expected answers

- Why predict before you run? Expected: A prediction shows whether you understand the line. A surprise is where learning happens.

- What does the comment in the code do? Expected: Nothing for Python. It helps the next person read the code.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Revenue by hand and by Python: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Reading an error message: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Revenue by hand and by Python): Change the Pen to 5 units and find the new total.
2. Modify, Block B (Reading an error message): Fix the NameError by creating units = 4 first and then printing units * 2000.
3. Make: Convert the text "12.5" to a number and calculate the price of four items at that price. Predict the result before you run it. Write one sentence on why division by zero cannot be answered.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: Why predict before you run? Expected: A prediction shows whether you understand the line. A surprise is where learning happens.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA04: Conditions and decisions

Private. Keep this file out of the student folder.

**Objective.** By the end: you can write a decision that gives a different answer for True and False, and test it at the edge.

**Question of the day.** Is this sale big enough to count as a large order?

> **WHERE THIS IS USED**
> Shops and delivery businesses often treat big orders differently, with free delivery, a discount or a call from the manager. A program makes that choice with if and else. What rule would you set for the big orders of your own shop?

Prerequisite: DA03 Arithmetic, revenue and reading errors.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

## Block A. Compare and decide

A comparison asks a question. == asks whether two values are equal. >= means at least. The word if runs its indented lines only when the answer is True. A colon ends the if line and four spaces mark the lines that belong to it.

Ask for the prediction first: Predict what prints for a budget of 4999, then for 5000, then for 5001.

```
budget = 5000
print(budget >= 5000)
if budget >= 5000:
    print("Buy")
else:
    print("Save")
```

Correct result: The cell prints True and then Buy. With a budget of 4999 the comparison is False and the cell prints Save. 5000 and 5001 both print Buy, because >= includes the edge value.

Investigate, with answers:

- Why does 5000 print Buy? Answer: >= means greater than or equal to, so the edge value passes.
- What is the difference between = and ==? Answer: = stores a value under a name. == asks whether two values are equal.

Modify: Set budget to 4999 and run the cell again. Answer: It prints Save.

```
budget = 4999
result = "Buy" if budget >= 5000 else "Save"
```

## Block B. Two conditions together

A large order needs 5 units or more. An expensive item costs 2000 shillings or more. and needs both to be True. or needs at least one.

Ask for the prediction first: Predict the answer for 6 units of an item at 1000 shillings, for both rules.

```
units = 6
price = 1000
large = units >= 5
expensive = price >= 2000
print(large and expensive)
print(large or expensive)
if large and expensive:
    print("Large and expensive")
elif large:
    print("Large only")
else:
    print("Small")
```

Correct result: The cell prints False, then True, then Large only. The order is large and not expensive, so and gives False and or gives True. elif means else if: it tests a second condition when the first is False.

Investigate, with answers:

- Why is the answer to large and expensive False? Answer: and needs both conditions True. The price is 1000, which is under 2000.
- Which branch runs for 3 units at 2500 shillings? Answer: Neither if nor elif, so else runs and it prints Small.

Modify: Change the price to 2500 and run the cell again. Answer: It prints Large and expensive.

```
price = 2500
expensive = price >= 2000
both = large and expensive
```

## Make: reference

```
def decide(budget):
    if budget >= 5000:
        return "Buy"
    else:
        return "Save"
print(decide(4999), decide(5000), decide(5001))
# Save Buy Buy
```

Build log entry to look for: DA04.ipynb with the decision tested at 4999, 5000 and 5001.

## Misconceptions to listen for

- = and == are the same.
- The indentation is only for looks.
- or means exactly one of the two.

## Questions to ask, with expected answers

- Which three values do you test for a rule that says at least 5000? Expected: 4999, 5000 and 5001: just below, on and just above the edge.
- What happens to the else lines when the condition is True? Expected: They are skipped.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Compare and decide: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Two conditions together: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Compare and decide): Set budget to 4999 and run the cell again.
2. Modify, Block B (Two conditions together): Change the price to 2500 and run the cell again.
3. Make: Write a decision for a ticket that costs 5000. Print "Buy" if the budget is at least 5000 and otherwise print "Save". Test 4999, 5000 and 5001.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: Which three values do you test for a rule that says at least 5000? Expected: 4999, 5000 and 5001: just below, on and just above the edge.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA05: Lists, dictionaries and loops

Private. Keep this file out of the student folder.

**Objective.** By the end: you can store several values in a list, look one up by position, and use a loop to add them up.

**Question of the day.** How many units did the first six sales sell in total?

> **WHERE THIS IS USED**
> A shop has many sales, a league has many matches and a tailor has many orders. A loop handles every one of them without you typing each by hand. What would you loop over in your favourite sport?

Prerequisite: DA04 Conditions and decisions.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

## Block A. Lists and positions

The units of the first six sales in the shop table are 1, 2, 4, 7, 6 and 5. A list keeps them in order inside square brackets. Square brackets after a list name pick out one position. Position 0 is the first.

Ask for the prediction first: Predict what each print shows.

```
units = [1, 2, 4, 7, 6, 5]
print(units[0])
print(units[3])
print(len(units))
print(max(units))
```

Correct result: The cell prints 1, 7, 6 and 7. Position 0 holds the first value and position 3 holds the fourth. len counts the values and max finds the largest.

Investigate, with answers:

- Why does units[3] show 7 and not 4? Answer: Counting starts at 0, so position 3 is the fourth value.
- Which position holds the last value? Answer: Position 5, which is len(units) - 1.

Modify: Print the value at position 4. Answer: 6.

```
fifth = units[4]
```

## Block B. Dictionaries and a loop with a running total

A dictionary stores named pairs. A for loop repeats its indented lines for each value in a list. The name after for stands for the current value.

Ask for the prediction first: Predict the final total and the price of Pen.

```
units = [1, 2, 4, 7, 6, 5]
prices = {"Book": 3000, "Pen": 1000, "Juice": 2000}
print(prices["Pen"])
total_units = 0
for u in units:
    total_units = total_units + u
print(total_units)
```

Correct result: The cell prints 1000 and then 25. The loop starts the running total at 0 and adds each of the six values in turn: 1, 3, 7, 14, 20, 25.

Investigate, with answers:

- What does the line total_units = total_units + u do? Answer: It reads the current total, adds u and stores the answer back under the same name.
- What would happen if total_units = 0 sat inside the loop? Answer: The total would restart for each value and end as the last value only, which is 5.

Modify: Change the loop to print each value plus 1 and then print the final total. Answer: The loop prints 2, 3, 5, 8, 7, 6.

```
plus_one = []
for u in units:
    plus_one.append(u + 1)
```

## Make: reference

```
values = [2, 4, 6]
for v in values:
    print(v + 1)
item = {"name": "Book", "stock": 8}
print(item["stock"])
# 3, 5, 7 and then 8.
```

Build log entry to look for: DA05.ipynb with the loop output 3, 5, 7, the stock 8 and the explanation of position 0.

## Misconceptions to listen for

- The first value is at position 1.
- The loop variable keeps its old value when the loop ends.
- A dictionary has an order like a list.

## Questions to ask, with expected answers

- How many values does a list of six have, and what is the last index? Expected: Six values. The last index is 5.
- Why does the running total start at zero? Expected: Nothing has been added yet.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Lists and positions: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Dictionaries and a loop with a running total: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Lists and positions): Print the value at position 4.
2. Modify, Block B (Dictionaries and a loop with a running total): Change the loop to print each value plus 1 and then print the final total.
3. Make: Make a list [2, 4, 6]. Print each value plus 1 with a loop. Make a dictionary with the keys name and stock and print the stock. Explain why position 0 is the first.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: How many values does a list of six have, and what is the last index? Expected: Six values. The last index is 5.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA06: Functions and checks that fail loudly

Private. Keep this file out of the student folder.

**Objective.** By the end: you can write a function that returns a result, and use assert to check it on known cases.

**Question of the day.** Can we trust our revenue calculation on cases we already know the answer to?

> **WHERE THIS IS USED**
> Payment and tax software relies on functions that must give the right answer on cases where the answer is already known. A check that fails loudly protects every customer. What known case would you use to test a discount function?

Prerequisite: DA05 Lists, dictionaries and loops.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

Where it overruns: Students often confuse the definition with a call. Run the def cell, show nothing happens, then call it.

## Block A. A function and a check

A function starts with def. print only shows something on the screen. return sends a result back so other code can use it. A passing assert prints nothing at all. A failing one stops with an error.

Ask for the prediction first: Predict what the function returns for 3 units at 2000 shillings, and what a passing assert prints.

```
def revenue(units, price):
    return units * price

print(revenue(3, 2000))
assert revenue(3, 2000) == 6000
assert revenue(0, 2000) == 0
```

Correct result: The cell prints 6000. The two asserts print nothing, which means both pass. units and price are the parameters. 3 and 2000 are the arguments.

Investigate, with answers:

- What is the difference between return and print? Answer: return gives the value to the rest of the program. print only shows it.
- What does it mean that the asserts print nothing? Answer: The statements are True. Silence means the check passed.

Modify: Add a third assert that checks revenue(5, 1000) equals 5000. Answer: It passes silently.

```
assert revenue(5, 1000) == 5000
```

## Block B. Edge cases and a guard

A share is a part divided by a whole. Dividing by zero is an edge case. A function can guard against it with an if before it divides.

Ask for the prediction first: Predict the result of share(12, 20) and of share(5, 0).

```
def share(part, whole):
    if whole == 0:
        return 0
    return part / whole

print(share(12, 20))
print(share(5, 0))
assert share(12, 20) == 0.6
assert share(5, 0) == 0
```

Correct result: The cell prints 0.6 and then 0. When the whole is zero the function returns before it divides, so no error appears.

Investigate, with answers:

- Why does the function check the whole before it divides? Answer: Dividing by zero stops the program with an error. The check avoids it.
- Is returning 0 for share(5, 0) always the right answer? Answer: Not always. It is a choice that must be stated, because a share of nothing is undefined.

Modify: Add an assert that share(1, 4) equals 0.25. Answer: It passes silently.

```
assert share(1, 4) == 0.25
```

## Make: reference

```
def price_after_discount(price, percent):
    return price * (100 - percent) / 100
assert price_after_discount(2000, 0) == 2000
assert price_after_discount(2000, 10) == 1800
assert price_after_discount(2000, 100) == 0
```

Build log entry to look for: DA06.ipynb with the function and three passing asserts, rerun from the top.

## Misconceptions to listen for

- print and return do the same thing.

- Defining a function runs it.
- A silent assert means nothing happened.

## Questions to ask, with expected answers

- Which three inputs do you test for a discount function? Expected: Zero percent, an ordinary percent and 100 percent, which are the edges and a middle case.
- Why use assert when we can read the output ourselves? Expected: The computer checks every time, and a wrong answer stops the program loudly.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, A function and a check: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Edge cases and a guard: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (A function and a check): Add a third assert that checks revenue(5, 1000) equals 5000.
2. Modify, Block B (Edge cases and a guard): Add an assert that share(1, 4) equals 0.25.
3. Make: Write price_after_discount(price, percent) that returns the price after a percentage discount. Check it with assert for 0 percent, 10 percent and 100 percent on a price of 2000. Save the notebook and run all cells from the top.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: Which three inputs do you test for a discount function? Expected: Zero percent, an ordinary percent and 100 percent, which are the edges and a middle case.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA07: A table in Python: DataFrame, columns and reading a CSV

Private. Keep this file out of the student folder.

**Objective.** By the end: you can build a small table, read a CSV file into one, and say how many rows and columns it has and what each column holds.

**Question of the day.** What is in each table we have been given?

> WHERE THIS IS USED
> Till exports, clinic registers and school timetables all become tables in Python. This is the table every data analyst works with. Which exported file would you most like to open?

Prerequisite: DA06 Functions and checks that fail loudly.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

Where it overruns: Six words and two blocks. If Block B runs long, shorten the products table and keep the shop table.

## Block A. Build a tiny table

pandas is a library for tables. import pandas as pd loads it and gives it the short name pd. A dot after a name reaches something that belongs to it: sales.shape asks the table for its size. A shape of (3, 2) means three rows and two columns.

Ask for the prediction first: Predict the shape of the table before you run the cell.

```
import pandas as pd
sales = pd.DataFrame({"item": ["Book", "Pen", "Book"], "units": [2, 5, 3]})
print(sales)
print(sales.shape)
print(list(sales.columns))
```

Correct result: The cell prints a table of three rows and two columns, then (3, 2), then ['item', 'units']. The first column holds the item sold and the second holds the units.

Investigate, with answers:

- Which number in (3, 2) counts rows? Answer: The first one. The shape is rows first and then columns.

- Where do the column headers come from? Answer: From the keys of the dictionary we gave pandas.

Modify: Add a third column called price_ugx with the values 3000, 1000 and 3000, then print the shape. Answer: The shape becomes (3, 3).

```
sales["price_ugx"] = [3000, 1000, 3000]
```

## Block B. Read a CSV file

pd.read_csv reads a CSV file and returns a DataFrame. The path is where the file is. Here the file sits beside the notebook, so its name is enough. dtypes lists the type of each column.

Ask for the prediction first: Predict how many rows and columns camp_shop.csv has, and which columns hold text.

```
shop = pd.read_csv("camp_shop.csv")
print(shop.shape)
print(shop.dtypes)
products = pd.read_csv("sunrise_products.csv")
print(products.shape)
print(products.head(3))
```

Correct result: The shop table has shape (60, 5). date and item hold text. The other three columns are whole numbers. The Sunrise products table has shape (15, 5): fifteen products, each with an id, a name, a category and two prices.

Investigate, with answers:

- Which columns would you use to work out revenue? Answer: units and unit_price_ugx. Revenue is units times unit price.
- Why is date a text column here? Answer: pandas reads it as text until we ask it to treat it as a date, which is a later session.

Modify: Show the last three rows of the shop table with tail(3). Answer: Sales 58, 59 and 60.

```
last3 = shop.tail(3)
```

## Make: reference

```
import pandas as pd
sunrise = pd.read_csv("sunrise_sales.csv")
print(sunrise.shape)
print(list(sunrise.columns))
# 1513 rows and 6 columns. sale_date holds the date and unit_price_ugx the price of one
    unit.
```

Build log entry to look for: DA07.ipynb with shape (1513, 6), the six column names and two sentences naming the date and price columns.

## Misconceptions to listen for

- A CSV file is an Excel file.
- Reading a file changes the file.

- The shape lists columns first.

## Questions to ask, with expected answers

- What is the difference between a row and a column? Expected: A row is one record. A column is one kind of information about every record.
- What does read_csv give back? Expected: A DataFrame.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Build a tiny table: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Read a CSV file: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Build a tiny table): Add a third column called price_ugx with the values 3000, 1000 and 3000, then print the shape.
2. Modify, Block B (Read a CSV file): Show the last three rows of the shop table with tail(3).
3. Make: Read sunrise_sales.csv, the Sunrise Shops sales. Print its shape and its column names. Write which column holds the date and which holds the price of one unit.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: What is the difference between a row and a column? Expected: A row is one record. A column is one kind of information about every record.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA08: Select, filter and calculate a revenue column

Private. Keep this file out of the student folder.

**Objective.** By the end: you can pick a column, keep the rows that meet a condition, and add a revenue column.

**Question of the day.** How much money did the practice shop take in, and where did it come from?

> WHERE THIS IS USED
> A shop owner asks which sales were big and how much money they brought in. Picking out rows and adding a new column is how analysts answer that on their own data. Which rows of a shop table would you pick out first?

Prerequisite: DA07 A table in Python: DataFrame, columns and reading a CSV.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

## Block A. Select and filter

Read the practice shop. Square brackets with a column name select one column. A comparison inside square brackets keeps only the rows where it is True.

Ask for the prediction first: Predict how many of the 60 sales have 5 units or more.

```
import pandas as pd
shop = pd.read_csv("camp_shop.csv")
units = shop["units"]
big = shop[shop["units"] >= 5]
print(len(big))
print(units.sum())
```

Correct result: The cell prints 30 and then 277. shop["units"] selects one column. The comparison gives True or False for every row, and only the True rows stay. All 60 sales together sold 277 units.

Investigate, with answers:

- Is the filter deleting the other 30 rows? Answer: No. big is a new, smaller table. shop still has all 60 rows.

- Why can the sum of units not tell us the money earned? Answer: The three items have different prices, so units and money are different quantities.

Modify: Filter the Juice sales with 5 units or more and count them. Answer: Seven sales.

```
juice_big = shop[(shop["item"] == "Juice") & (shop["units"] >= 5)]
n_juice_big = len(juice_big)
```

## Block B. A new revenue column

Assigning to a new column name creates the column. Multiplying two columns multiplies their matching rows. This is how units times price becomes revenue.

Ask for the prediction first: Predict the total revenue, using the item prices 3000, 1000 and 2000.

```
shop["revenue_ugx"] = shop["units"] * shop["unit_price_ugx"]
print(shop.head(3))
print(shop["revenue_ugx"].sum())
```

Correct result: The first three revenues are 3000, 2000 and 8000. The total revenue is 550000 shillings. This is money taken in. There is no cost column, so profit cannot be worked out.

Investigate, with answers:

- Why is revenue not profit? Answer: Profit is revenue minus the cost of the goods. The table has no cost.
- Does the new column change camp_shop.csv? Answer: No. It exists only in the notebook until you save it.

Modify: Find the revenue of the Book sales only. Answer: 297000.

```
book_revenue = shop[shop["item"] == "Book"]["revenue_ugx"].sum()
```

## Make: reference

```
import pandas as pd
shop = pd.read_csv("camp_shop.csv")
shop["revenue_ugx"] = shop["units"] * shop["unit_price_ugx"]
pens = shop[(shop["item"] == "Pen") & (shop["units"] >= 4)]
print(len(pens), pens["revenue_ugx"].sum())
# 14 sales and 90000 shillings. The count is how many sales. The revenue is how much
    money.
```

Build log entry to look for: DA08.ipynb with 14 sales, 90000 shillings and a sentence on the difference.

## Misconceptions to listen for

- Filtering removes rows from the file.
- The item with the most units earns the most money.
- A new column is saved in the CSV automatically.

## Questions to ask, with expected answers

- Why do we use & and not the word and inside the brackets of a filter? Expected: pandas combines two columns of True and False values row by row, and & does that.
- Why should the total of the parts equal the whole? Expected: It is the first check that a calculation did not lose or double rows.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Select and filter: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, A new revenue column: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Select and filter): Filter the Juice sales with 5 units or more and count them.
2. Modify, Block B (A new revenue column): Find the revenue of the Book sales only.
3. Make: Keep the Pen sales that have 4 units or more. Count them and add up the revenue they bring in. Say why the count and the revenue are different quantities.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: Why do we use & and not the word and inside the brackets of a filter? Expected: pandas combines two columns of True and False values row by row, and & does that.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA09: Sort, group, mean and median

Private. Keep this file out of the student folder.

**Objective.** By the end: you can total a column by group, sort the result, and say when the median describes a column better than the mean.

**Question of the day.** Which item earns the most, and what is a typical sale in a small practice shop?

> WHERE THIS IS USED
> Retail chains compare their branches by grouping sales and finding the best sellers. A typical sale and an unusually large one tell different stories. Which branch would you visit first, and why?

Prerequisite: DA08 Select, filter and calculate a revenue column.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

Where it overruns: The move to the 1513-row table is the first time students cannot check every row by eye. Say so aloud and check totals instead.

## Block A. Group, total and sort

groupby gathers the rows with the same item. A calculation such as sum then works inside each group. sort_values puts the answer in order.

Ask for the prediction first: Predict which item sold the most units and which earned the most money.

```
import pandas as pd
shop = pd.read_csv("camp_shop.csv")
shop["revenue_ugx"] = shop["units"] * shop["unit_price_ugx"]
units_by_item = shop.groupby("item")["units"].sum()
revenue_by_item = shop.groupby("item")["revenue_ugx"].sum().sort_values(ascending=False)
print(units_by_item)
print(revenue_by_item)
```

Correct result: Pen sold the most units, 103. Book earned the most money, 297000 shillings, then Juice 150000 and Pen 103000. The three revenues add to 550000. The item that sells the most units is not the item that earns the most.

Investigate, with answers:

- Why does Pen sell the most units and earn the least money? Answer: A Pen costs 1000 shillings and a Book costs 3000, so each Pen sale brings in less.
- How do you check the grouped totals? Answer: Add them up and compare with the overall total of 550000.

Modify: Group the revenue by date and find the date with the most revenue. Answer: 2026-12-20 with 47000.

```
by_date = shop.groupby("date")["revenue_ugx"].sum()
best_date = by_date.idxmax()
```

## Block B. Mean, median and a proportion

Now meet the bigger Sunrise Shops table: 1513 sales from three shops. The mean can be pulled up by a few huge sales. The median is not. A proportion needs its whole named: 36 large orders out of 1513 sales.

Ask for the prediction first: Predict whether the mean sale value is above or below the median.

```
sunrise = pd.read_csv("sunrise_sales.csv")
sunrise["line_value"] = sunrise["units"] * sunrise["unit_price_ugx"]
mean_value = sunrise["line_value"].mean()
median_value = sunrise["line_value"].median()
bulk = (sunrise["units"] >= 10).sum()
share = bulk / len(sunrise)
print(round(mean_value), median_value)
print(bulk, len(sunrise), round(share * 100, 1))
```

Correct result: The mean sale value is about 4214 shillings and the median is 2400. A few bulk purchases pull the mean up. 36 of the 1513 sales have 10 units or more, which is 2.4 percent. The 1513 is the denominator.

Investigate, with answers:

- Which number is a better answer to "what does a typical sale bring in"? Answer: The median, 2400, because a few bulk sales inflate the mean.
- Why must you say that the whole is 1513 sales? Answer: A percentage with no stated whole cannot be compared with another percentage.

Modify: Find the proportion of sales with 5 units or more. Answer: About 8.7 percent, 132 of 1513.

```
n5 = (sunrise["units"] >= 5).sum()
share5 = n5 / len(sunrise)
```

## Make: reference

```
import pandas as pd
sunrise = pd.read_csv("sunrise_sales.csv")
sunrise["line_value"] = sunrise["units"] * sunrise["unit_price_ugx"]
by_branch = sunrise.groupby("branch_id")["line_value"].sum().sort_values(ascending=False)
print(by_branch)
assert by_branch.sum() == sunrise["line_value"].sum()
# Branch 1: 2997000. Branch 2: 1985400. Branch 3: 1393500. Total 6375900.
```

Build log entry to look for: DA09.ipynb with three sorted totals that add to 6375900 and a passing assert.

## Misconceptions to listen for

- The item that sells the most units earns the most money.
- The mean is always the best summary.
- A percentage is meaningful without its whole.

## Questions to ask, with expected answers

- When would you quote the median and not the mean? Expected: When a few very large values pull the mean away from what most sales look like.
- What check follows every group total? Expected: Add the groups and compare with the overall total.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Group, total and sort: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Mean, median and a proportion: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Group, total and sort): Group the revenue by date and find the date with the most revenue.
2. Modify, Block B (Mean, median and a proportion): Find the proportion of sales with 5 units or more.
3. Make: Add the line_value column and total it for each branch_id in the Sunrise sales. Sort the three totals from largest to smallest. Check that the three totals add up to the overall total.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: When would you quote the median and not the mean? Expected: When a few very large values pull the mean away from what most sales look like.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA10: Cleaning without hiding evidence

Private. Keep this file out of the student folder.

**Objective.** By the end: you can find missing values and duplicates, remove the duplicates on a copy, and save a cleaning log that says what you did.

**Question of the day.** What is wrong with the raw Sunrise sales file, and how many rows does each problem touch?

> WHERE THIS IS USED
> Exports from tills often hold duplicates, blanks and typing slips. Good analysts keep a log of every fix so the owner can trust the result. How many problems can you find before you fix a single one?

Prerequisite: DA09 Sort, group, mean and median.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

## Block A. Missing values and duplicates in a small table

Keep the raw table unchanged and work on a copy. isna marks missing cells True. Adding the True values counts them. drop_duplicates removes rows that repeat another row exactly.

Ask for the prediction first: Predict how many missing prices there are and how many rows stay after the duplicate is removed.

```
import pandas as pd
raw = pd.DataFrame({"id": [1, 2, 2, 3], "price": [1000, None, None, 3000]})
clean = raw.drop_duplicates().copy()
print(raw["price"].isna().sum())
print(len(clean))
print(clean["price"].mean())
```

Correct result: The raw table has 2 missing prices. After the duplicate row is removed, 3 rows stay and one price is still missing. The mean of the two known prices is 2000. The missing price is unknown and is not free.

Investigate, with answers:

- Why would replacing the missing price with 0 change the meaning? Answer: Zero says the item was free. The truth is that nobody recorded the price.
- Could two identical rows be real separate sales? Answer: Yes, if two customers bought the same thing. Here the repeated id shows it is a copy.

Modify: Count how many rows in clean still have a missing price. Answer: One.

```
still_missing = clean["price"].isna().sum()
```

## Block B. The raw Sunrise file

sunrise_sales_raw.csv is the Sunrise sales as an untidy export. Count the problems. to_csv saves a table to a file. index=False stops pandas adding an extra column of row numbers.

Ask for the prediction first: Predict how many duplicate rows the file holds. The clean version has 1513 sales.

```
raw_sales = pd.read_csv("sunrise_sales_raw.csv")
print(raw_sales.shape)
print(raw_sales.duplicated().sum())
deduped = raw_sales.drop_duplicates()
print(deduped.shape)
print(deduped["units"].isna().sum())
log = pd.DataFrame({"problem": ["duplicate rows", "blank units"], "rows":
    [raw_sales.duplicated().sum(), deduped["units"].isna().sum()]})
log.to_csv("cleaning_log.csv", index=False)
print(pd.read_csv("cleaning_log.csv"))
```

Correct result: The raw file has 1538 rows. 25 are exact duplicates, so removing them leaves 1513, the same as the clean file. 40 rows have blank units. The log table is saved and read back with two lines: 25 duplicate rows and 40 blank-unit rows.

Investigate, with answers:

- Why does the cleaned row count of 1513 match the clean file? Answer: The 25 extra rows were exact copies. Removing them restores the original count.
- Why not fill the 40 blank units with the mean? Answer: It would invent sales that nobody recorded. We decide in session 16 how to report them.

Modify: Count the rows whose date is written with slashes, and add them to the log. Answer: 60 rows.

```
slashes = deduped["sale_date"].str.contains("/").sum()
```

## Make: reference

```
import pandas as pd
log = pd.DataFrame({"step": ["raw rows", "duplicate removed", "missing price remaining"],
    "count": [4, 1, 1]})
log.to_csv("my_log.csv", index=False)
print(pd.read_csv("my_log.csv"))
# Zero would say the item was free. The price is unknown.
```

Build log entry to look for: DA10.ipynb and my_log.csv with the three counts and the sentence about zero.

## Misconceptions to listen for

- A missing value is the same as zero.
- Every repeated row is an error.
- Cleaning means making the numbers look nice.

## Questions to ask, with expected answers

- What goes in a cleaning log? Expected: The problem, how many rows it touched, and what you did.
- Why work on a copy? Expected: The raw file is the evidence. Overwriting it destroys the evidence.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Missing values and duplicates in a small table: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, The raw Sunrise file: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Missing values and duplicates in a small table): Count how many rows in clean still have a missing price.
2. Modify, Block B (The raw Sunrise file): Count the rows whose date is written with slashes, and add them to the log.
3. Make: Write a cleaning log for the four-row table in Block A: four raw rows, one duplicate removed, one price still missing. Save it as a CSV with to_csv. Write one sentence on why replacing the missing price with zero would change what the data says.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: What goes in a cleaning log? Expected: The problem, how many rows it touched, and what you did.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA11: Charts that say what the numbers say

Private. Keep this file out of the student folder.

**Objective.** By the end: you can draw a bar chart, a line chart, a histogram and a scatter plot with labelled axes, and state one finding and one limit for each.

**Question of the day.** Which chart shows this comparison most clearly?

> **WHERE THIS IS USED**
> Weather pages, football stats and mobile money summaries all lean on a chart to make one point quickly. A busy shop owner has a minute at most. Which chart would you show an owner who has only one minute?

Prerequisite: DA10 Cleaning without hiding evidence.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

Where it overruns: plt.show() blocks nothing in a notebook, but charts can overlap. Close each figure before the next.

## Block A. Bar and line charts

plt.bar draws one bar for each category. A bar chart starts its value axis at zero, because the length of a bar stands for the amount. plt.plot joins points in order, which suits dates.

Ask for the prediction first: Predict which of the three bars will be tallest.

```
import pandas as pd
import matplotlib.pyplot as plt
shop = pd.read_csv("camp_shop.csv")
shop["revenue_ugx"] = shop["units"] * shop["unit_price_ugx"]
by_item = shop.groupby("item")["revenue_ugx"].sum()
plt.bar(by_item.index, by_item.values)
plt.ylabel("Revenue (UGX)")
plt.title("Revenue by item, practice shop")
plt.ylim(bottom=0)
plt.show()
by_date = shop.groupby("date")["revenue_ugx"].sum()
plt.plot(by_date.index, by_date.values)
plt.ylabel("Revenue (UGX)")
plt.xticks(rotation=90)
```

```
plt.show()
```

Correct result: The first chart has three bars. Book is tallest at 297000, then Juice and Pen. The second chart is a line of 20 daily revenues, with the highest point on 2026-12-20. Every axis says what it measures and the bar chart starts at zero.

Investigate, with answers:

- Why does the bar chart start at zero? Answer: The length of the bar stands for the amount. A cut-off axis makes small differences look huge.
- Why use a line for dates and bars for items? Answer: Dates have an order, so a line shows change. Items have no order, so separate bars are fairer.

Modify: Draw the bar chart of units by item and add its label. Answer: Pen is tallest with 103 units.

```
units_by_item = shop.groupby("item")["units"].sum()
plt.bar(units_by_item.index, units_by_item.values)
plt.ylabel("Units sold")
plt.close("all")
```

## Block B. Histogram and scatter plot

A histogram shows how the values are spread. A scatter plot puts two numbers on the two axes. These two charts use the Sunrise Shops tables.

Ask for the prediction first: Predict the shape of the histogram of sale values: balanced, or bunched at the low end with a long tail?

```
sunrise = pd.read_csv("sunrise_sales.csv")
sunrise["line_value"] = sunrise["units"] * sunrise["unit_price_ugx"]
plt.hist(sunrise["line_value"], bins=30)
plt.xlabel("Value of one sale (UGX)")
plt.ylabel("Number of sales")
plt.show()
daily = pd.read_csv("sunrise_daily_summary.csv")
plt.scatter(daily["customers_counted"], daily["revenue_ugx"])
plt.xlabel("Customers counted in a day")
plt.ylabel("Revenue that day (UGX)")
plt.show()
```

Correct result: The histogram is bunched at the low end with a long tail to the right, because most sales are small and a few are very large. The scatter plot has 180 dots, one for each shop and day. More customers usually mean more revenue, but the dots are scattered, so the link is not exact.

Investigate, with answers:

- What does the long tail in the histogram tell us? Answer: A few sales are far larger than the rest. That is why the mean is above the median.
- Does the scatter plot show that customers cause revenue? Answer: It shows that they go together. It does not prove cause. That needs more evidence.

Modify: Change bins=30 to bins=10 and describe what the histogram loses. Answer: It hides detail in the long tail.

```
counts, edges = plt.hist(sunrise["line_value"], bins=10)[:2]
plt.close("all")
```

## Make: reference

```
import pandas as pd
import matplotlib.pyplot as plt
sunrise = pd.read_csv("sunrise_sales.csv")
sunrise["line_value"] = sunrise["units"] * sunrise["unit_price_ugx"]
names = {1: "Central Market", 2: "Campus Gate", 3: "Roadside"}
sunrise["branch"] = sunrise["branch_id"].map(names)
by_branch = sunrise.groupby("branch")["line_value"].sum()
plt.bar(by_branch.index, by_branch.values)
plt.ylabel("Revenue (UGX)")
plt.ylim(bottom=0)
plt.show()
# Central Market earns the most, 2997000. The data is invented, so it says nothing about
    real shops.
```

Build log entry to look for: DA11.ipynb with a labelled bar chart of three branches, one finding and one limitation.

## Misconceptions to listen for

- A line chart suits any categories.
- A cut-off axis makes the chart clearer.
- Two things that move together cause each other.

## Questions to ask, with expected answers

- Which chart suits the revenue of each branch, and why? Expected: A bar chart, because the branches are separate categories.
- What must every chart carry? Expected: Labelled axes with units, and a title that says what is shown.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Bar and line charts: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Histogram and scatter plot: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Bar and line charts): Draw the bar chart of units by item and add its label.
2. Modify, Block B (Histogram and scatter plot): Change bins=30 to bins=10 and describe what the histogram loses.
3. Make: Chart the Sunrise revenue by branch. Use a dictionary to turn the ids 1, 2 and 3 into the names Central Market, Campus Gate and Roadside. Label the axis, start it at zero and write one finding and one limitation.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: Which chart suits the revenue of each branch, and why? Expected: A bar chart, because the branches are separate categories.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA12: Show your first chart and explain what it says

Private. Keep this file out of the student folder.

**Objective.** By the end: you can read a table, check it, summarise it, chart it and explain two findings and one limit, from a restarted notebook.

**Question of the day.** What do the Sunrise Shops sales tell us about the three shops?

> **WHERE THIS IS USED**
> Every analyst sends someone a first report: a table, one chart and a few honest sentences. Today you make yours. Who would you most like to send it to?

Prerequisite: DA11 Charts that say what the numbers say.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 15 | recall, then read the brief together |
| 80 | learners work and the instructor visits each of the five |
| 15 | break away from screens |
| 40 | learners finish, restart and run all |
| 20 | a quiet one-to-one chat, then save and share |
| 10 | review |

Where it overruns: Do not teach new syntax. Repair with the error cards from sessions 7 to 11.

## Make: reference

```
import pandas as pd
import matplotlib.pyplot as plt
sales = pd.read_csv("sunrise_sales.csv")
sales["line_value"] = sales["units"] * sales["unit_price_ugx"]
rows = len(sales)
total = sales["line_value"].sum()
missing = int(sales.isna().sum().sum())
duplicates = int(sales.duplicated().sum())
names = {1: "Central Market", 2: "Campus Gate", 3: "Roadside"}
sales["branch"] = sales["branch_id"].map(names)
by_branch = sales.groupby("branch")["line_value"].sum().sort_values(ascending=False)
assert by_branch.sum() == total
plt.bar(by_branch.index, by_branch.values)
plt.ylabel("Revenue (UGX)")
plt.ylim(bottom=0)
plt.show()
# 1513 rows, 6375900 in revenue, no missing values, no duplicates. Central Market earns
    the most.
```

Build log entry to look for: DA12.ipynb that reruns from a restarted kernel, with the rows, total, checks, sorted branch totals, one labelled chart, two findings and one limitation. It goes in the build log, and the instructor gives one strength and one next step in a private chat.

## Misconceptions to listen for

- A notebook that works now will work after a restart.
- More charts show more skill.
- A finding needs no number.

## Questions to ask, with expected answers

- Where does each number in your findings come from? Expected: A named output cell. If it has no cell, it does not go in the finding.
- What can this data not tell us? Expected: Anything about real shops. It is invented, and it has no costs, so no profit.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Read the brief together on the shared screen, then the first build step. Each learner shares their screen in turn. |
| 5 | Break away from the screen |
| 35 | Learners keep building. The instructor visits each learner by screen share, about seven minutes each. |
| 10 | Close: say the exit sentence aloud, then the rest of the build task is set |

### After-class build task, about an hour

1. Make: In DA12.ipynb use sunrise_sales.csv. 1) Report the number of rows and the total revenue. 2) Check for missing values and duplicates and report the counts. 3) Total the revenue by branch with names from a dictionary and sort the result. 4) Draw one labelled chart. 5) Write two findings and one limitation in text cells. 6) Look at the topic on your progress map and write one project question that this table, or another supplied table, could help answer. Restart the kernel, run all cells from the top, save DA12.ipynb with a short report.pdf or report.docx, and share them by the route in START_HERE.md.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: Where does each number in your findings come from? Expected: A named output cell. If it has no cell, it does not go in the finding.

This is a work session. Keep the live part for the brief and for individual check-ins. The learner builds in the live time and after class.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA13: NumPy: the numbers underneath pandas

Private. Keep this file out of the student folder.

**Objective.** By the end: you can turn a column into a NumPy array, calculate its mean, median, spread and percentiles, and find unusually large values.

**Question of the day.** How spread out are the daily customer counts, and which sales are unusually large?

> **WHERE THIS IS USED**
> Weather offices and football analysts work with long rows of numbers, such as daily rainfall or distance run. They check how spread out the numbers are and spot unusual days. Which day in a shop would you call unusual?

Prerequisite: DA12 Show your first chart and explain what it says.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

Where it overruns: np.percentile with a list of two values returns an array. Print it before using it.

## Block A. Arrays, summaries and percentiles

Each shop counted its customers every day for 60 days: 180 counts in all. .to_numpy() turns a column into an array. np.mean and np.median give the usual summaries and np.percentile gives the value below which a share of the counts fall.

Ask for the prediction first: Predict whether the mean or the median of the daily counts is bigger.

```
import numpy as np
import pandas as pd
daily = pd.read_csv("sunrise_daily_summary.csv")
customers = daily["customers_counted"].to_numpy()
print(customers.shape)
print(round(np.mean(customers), 1), np.median(customers))
print(np.percentile(customers, [25, 75]))
print(customers.max() - customers.min())
print(np.mean(customers) == daily["customers_counted"].mean())
```

Correct result: The array has shape (180,). The mean is 143.0 and the median 131.5, so the mean is bigger. The 25th and 75th percentiles are 86.75 and 203.75: a quarter of the days had fewer than 87 customers and a quarter had more than 204. The range is 332. The last line prints True, because pandas

and NumPy give the same mean.

Investigate, with answers:

- Why are the mean and the median different here? Answer: Three shops of very different sizes are in the same column. The busy shop pulls the mean up.
- What do the two percentiles tell the owner? Answer: Half of all shop-days fall between 87 and 204 customers. That is the usual range of a day.

Modify: Add 50 to every count at once and print the new mean. Answer: 193.0.

```
busier = customers + 50
```

## Block B. Spread and outliers

The standard deviation says how far values usually sit from their mean. A common rule marks a value as an outlier when it is more than 1.5 times the gap between the quartiles above the upper quartile.

Ask for the prediction first: Predict how many of the 1513 sales are unusually large by that rule.

```
sunrise = pd.read_csv("sunrise_sales.csv")
values = (sunrise["units"] * sunrise["unit_price_ugx"]).to_numpy()
q1, q3 = np.percentile(values, [25, 75])
fence = q3 + 1.5 * (q3 - q1)
big = values[values > fence]
print(q1, q3, fence)
print(len(big), big.max())
print(round(np.std(customers), 1))
```

Correct result: The quartiles are 1200 and 4500, so the fence is 9450. 137 of the 1513 sales are above it, and the largest is 108000. The standard deviation of the daily customer counts is 73.7. A few large sales sit far out. They are real sales, so we report them and do not delete them.

Investigate, with answers:

- Should we delete the 137 outliers? Answer: No. They are real sales, mostly bulk purchases. We report them and say that they pull the mean up.
- What does a standard deviation of 73.7 customers tell us? Answer: Daily counts typically sit about 74 customers from the mean of 143.

Modify: Count the sales with a value above 20000. Answer: A small number that stays unusual.

```
n_huge = int((values > 20000).sum())
```

## Make: reference

```
import numpy as np
import pandas as pd
daily = pd.read_csv("sunrise_daily_summary.csv")
revenue = daily["revenue_ugx"].to_numpy()
np_route = [np.mean(revenue), np.median(revenue)] + list(np.percentile(revenue, [25,
    75]))
pd_route = [daily["revenue_ugx"].mean(), daily["revenue_ugx"].median(),
    daily["revenue_ugx"].quantile(0.25), daily["revenue_ugx"].quantile(0.75)]
assert np.allclose(np_route, pd_route)
print(np_route)
# Mean 35421.67, median 30350, quartiles 18400 and 44125.
```

Build log entry to look for: DA13.ipynb with the four numbers from both routes and a passing allclose check.

## Misconceptions to listen for

- An outlier is a mistake and should be deleted.
- NumPy and pandas give different answers.
- The mean and median should be equal.

## Questions to ask, with expected answers

- Why do we not delete outliers automatically? Expected: They may be real. Deleting them changes the story without telling the reader.
- What does np.percentile(x, 75) return? Expected: The value that 75 percent of the numbers fall below.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Arrays, summaries and percentiles: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Spread and outliers: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Arrays, summaries and percentiles): Add 50 to every count at once and print the new mean.

2. Modify, Block B (Spread and outliers): Count the sales with a value above 20000.
3. Make: Take the revenue_ugx column of the daily summary as a NumPy array. Find its mean, median and the 25th and 75th percentiles with NumPy. Then find the same four numbers with pandas and check that the two routes agree.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: Why do we not delete outliers automatically? Expected: They may be real. Deleting them changes the story without telling the reader.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA14: Exploring distributions with seaborn

Private. Keep this file out of the student folder.

**Objective.** By the end: you can draw a histogram and a box plot with seaborn and describe the shape, the middle and the outliers of a column.

**Question of the day.** What does a typical sale look like, and does it differ between the three shops?

> **WHERE THIS IS USED**
> A histogram shows what a typical day looks like, and a box plot shows how groups differ. Club analysts compare players and teams with plots like these. Which two shops would you compare first?

Prerequisite: DA13 NumPy: the numbers underneath pandas.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

## Block A. A histogram of sale values

sns.histplot draws a histogram from a DataFrame. Name the DataFrame in data= and the column in x=. A map turns branch ids into names.

Ask for the prediction first: Predict whether the histogram is balanced or skewed to the right.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
sunrise = pd.read_csv("sunrise_sales.csv")
sunrise["line_value"] = sunrise["units"] * sunrise["unit_price_ugx"]
sunrise["branch"] = sunrise["branch_id"].map({1: "Central Market", 2: "Campus Gate", 3:
    "Roadside"})
sns.histplot(data=sunrise, x="line_value", bins=30)
plt.xlabel("Value of one sale (UGX)")
plt.show()
print(round(sunrise["line_value"].skew(), 1))
```

Correct result: The histogram is bunched at the low end with a long tail to the right. The skew is 7.3. A positive skew above 1 means a long right tail. Most sales are small and a few are large.

Investigate, with answers:

- What does the long right tail mean for the mean and the median? Answer: The tail pulls the mean above the median. The median is the better description of a typical sale.
- What does exploratory analysis do that a single total does not? Answer: It shows the shape of the data, so we see the bulk purchases that one total hides.

Modify: Draw the histogram again with bins=60 and say what changes. Answer: The bars get narrower and the long tail shows more detail.

```
sns.histplot(data=sunrise, x="line_value", bins=60)
plt.close("all")
```

## Block B. A box plot for each shop

sns.boxplot draws one box for each group. The line inside the box is the median. The box runs from the lower quartile to the upper quartile. Dots beyond the whiskers are outliers.

Ask for the prediction first: Predict which shop has the lowest median sale.

```
sns.boxplot(data=sunrise, x="branch", y="line_value")
plt.ylabel("Value of one sale (UGX)")
plt.show()
medians = sunrise.groupby("branch")["line_value"].median()
print(medians)
```

Correct result: The medians are 2500 at Central Market, 2000 at Campus Gate and 2500 at Roadside. The boxes look alike and every shop has dots well above its whiskers: bulk sales happen in all three. The shops differ more in how many sales they make than in how big a typical sale is.

Investigate, with answers:

- Do the shops differ much in typical sale value? Answer: Not much. Two medians are 2500 and one is 2000.
- What do the dots above each box show? Answer: Unusually large sales. They exist at all three shops.

Modify: Use the box plot to compare the units column across shops. Print the median units for each shop. Answer: A similar median at each shop.

```
unit_medians = sunrise.groupby("branch")["units"].median()
```

## Make: reference

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
daily = pd.read_csv("sunrise_daily_summary.csv")
sns.histplot(data=daily, x="customers_counted", bins=20)
plt.show()
sns.boxplot(data=daily, x="branch", y="customers_counted")
plt.show()
med = daily.groupby("branch")["customers_counted"].median()
print(med)
# Central Market 225, Campus Gate 136, Roadside 87. Central Market has more than twice
    the customers of Roadside on a typical day.
```

Build log entry to look for: DA14.ipynb with a histogram, a box plot, three medians and a finding.

## Misconceptions to listen for

- A box plot shows every value.
- The line in the box is the mean.
- Outlier dots mean the data is wrong.

## Questions to ask, with expected answers

- What does the line in a box plot show? Expected: The median.
- How do you see a skewed distribution in a histogram? Expected: The bars bunch at one end and trail off in a long tail at the other.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, A histogram of sale values: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, A box plot for each shop: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (A histogram of sale values): Draw the histogram again with bins=60 and say what changes.
2. Modify, Block B (A box plot for each shop): Use the box plot to compare the units column across shops. Print the median units for each shop.
3. Make: Explore the daily customer counts. Draw a histogram of customers_counted and a box plot of customers_counted for each branch. Write the median for each branch and one finding in a sentence.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: What does the line in a box plot show? Expected: The median.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA15: Exploring relationships: scatter plots, correlation and cause

Private. Keep this file out of the student folder.

**Objective.** By the end: you can draw a scatter plot coloured by group, read a correlation number, and say why a correlation does not prove one thing causes another.

**Question of the day.** Do busier days bring in more money, and does rain keep customers away?

> **WHERE THIS IS USED**
> Taxi parks, shops and farms all ask whether rain changes demand. Two things moving together do not prove that one causes the other. Do rainy days keep customers away, or is something else going on?

Prerequisite: DA14 Exploring distributions with seaborn.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

## Block A. A scatter plot and one correlation

sns.scatterplot puts one column on each axis. hue= colours the dots by group. .corr() calculates the correlation between two columns.

Ask for the prediction first: Predict whether the correlation between customers and revenue is close to 0, close to 1 or close to -1.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
daily = pd.read_csv("sunrise_daily_summary.csv")
sns.scatterplot(data=daily, x="customers_counted", y="revenue_ugx", hue="branch")
plt.show()
r = daily["customers_counted"].corr(daily["revenue_ugx"])
print(round(r, 3))
```

Correct result: The dots rise from left to right, so more customers usually mean more revenue. The correlation is 0.623: positive and moderately strong. The dots are scattered, so a customer count does not fix the revenue.

Investigate, with answers:

- What would a correlation of 1 look like on the chart? Answer: All the dots would lie exactly on one rising line.
- Why do the colours help? Answer: They show that the three shops sit in different parts of the chart. Central Market is at the busy end.

Modify: Find the correlation between customers and the number of sale lines (sale_lines). Answer: 0.744, stronger.

```
r_lines = daily["customers_counted"].corr(daily["sale_lines"])
```

## Block B. A heatmap of every pair, and the rain question

A heatmap of .corr() shows every pair at once. annot=True writes the numbers in the squares. Rain is in the data as rain_mm. Does it matter?

Ask for the prediction first: Predict whether rain_mm is strongly related to customers.

```
cols = ["customers_counted", "rain_mm", "sale_lines", "revenue_ugx"]
table = daily[cols].corr()
sns.heatmap(table, annot=True, vmin=-1, vmax=1)
plt.show()
print(round(table.loc["rain_mm", "customers_counted"], 3))
print(round(table.loc["sale_lines", "revenue_ugx"], 3))
```

Correct result: Rain and customers have a correlation of -0.013, which is essentially zero. Sale lines and revenue have 0.565. In this data the weather barely matters. A weak correlation is a finding too.

Investigate, with answers:

- What does -0.013 tell us about rain? Answer: Almost nothing links them in this data. Wet days and dry days look alike.
- Could a strong correlation still be a coincidence? Answer: Yes. Two things can move together without one causing the other, so we look for a reason as well.

Modify: Find the correlation of rain_mm with revenue_ugx. Answer: -0.054, also close to zero.

```
r_rain = table.loc["rain_mm", "revenue_ugx"]
```

## Make: reference

```
import pandas as pd
daily = pd.read_csv("sunrise_daily_summary.csv")
result = {}
for name in ["Central Market", "Campus Gate", "Roadside"]:
    part = daily[daily["branch"] == name]
    result[name] = round(part["rain_mm"].corr(part["customers_counted"]), 3)
print(result)
# All three are weak. The largest is Roadside at -0.159. Even that does not prove that
    rain causes fewer customers.
```

Build log entry to look for: DA15.ipynb with three correlations and a sentence on why correlation is not cause.

## Misconceptions to listen for

- A strong correlation proves cause.
- A correlation near zero means the chart is wrong.
- Correlation of -0.5 means no relationship.

## Questions to ask, with expected answers

- What would you check before saying rain keeps customers away? Expected: A reason and more evidence: the correlation is weak, and other things change from day to day.
- What does a negative correlation mean? Expected: One goes up as the other goes down.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, A scatter plot and one correlation: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, A heatmap of every pair, and the rain question: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (A scatter plot and one correlation): Find the correlation between customers and the number of sale lines (sale_lines).
2. Modify, Block B (A heatmap of every pair, and the rain question): Find the correlation of rain_mm with revenue_ugx.
3. Make: Check the rain question for each shop separately. For each branch find the correlation between rain_mm and customers_counted. Say whether rain matters at any shop and why a correlation could not prove that it does.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: What would you check before saying rain keeps customers away? Expected: A reason and more evidence: the correlation is weak, and other things change from day to day.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA16: Cleaning the raw export

Private. Keep this file out of the student folder.

**Objective.** By the end: you can turn a messy export into a clean table with a pipeline that reruns, and explain why its total is lower than the true total.

**Question of the day.** If we clean the raw export carefully, does it give back the true revenue?

> WHERE THIS IS USED
> A raw export from a real system is rarely tidy. Analysts clean it with a function so the same steps run again on next month's file. Could you clean a file you have never seen?

Prerequisite: DA15 Exploring relationships: scatter plots, correlation and cause.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

Where it overruns: Two blocks of dense code. Run each line in its own cell the first time, and print after each.

## Block A. Dates and names

The raw export has two date formats in one column: 2026-09-01 and 01/09/2026. Parse each format on its own, and keep the date that worked. Names appear as juice, JUICE and " Juice ". Strip the spaces, fix the case and look each name up in the products table.

Ask for the prediction first: Predict how many dates cannot be parsed after both formats have been tried.

```
import pandas as pd
raw = pd.read_csv("sunrise_sales_raw.csv")
work = raw.drop_duplicates().copy()
iso = pd.to_datetime(work["sale_date"], format="%Y-%m-%d", errors="coerce")
slash = pd.to_datetime(work["sale_date"], format="%d/%m/%Y", errors="coerce")
work["sale_date"] = iso.fillna(slash)
print(work["sale_date"].isna().sum())
print(work["sale_date"].min().date(), work["sale_date"].max().date())
products = pd.read_csv("sunrise_products.csv")
lookup = {n.title(): n for n in products["name"]}
work["product"] = work["product"].str.strip().str.title().map(lookup)
print(work["product"].isna().sum(), work["product"].nunique())
```

Correct result: The cell prints 0 dates that cannot be parsed, then the range 2026-09-01 to 2026-10-30. The names step prints 0 names left unmatched and 15 different products. Every date and product name is now one consistent form.

Investigate, with answers:

- Why parse the two formats separately and join the results? Answer: Python cannot tell whether 03/09/2026 is 3 September or 9 March. We know the second format is day first, so we say so.
- What does errors="coerce" do? Answer: It turns a date that does not fit the format into a missing value instead of stopping with an error.

Modify: Count how many rows used the slash format. Answer: 60.

```
n_slash = int(raw.drop_duplicates()["sale_date"].str.contains("/").sum())
```

## Block B. Units, blanks and the residual

Some units are written as words, such as two. A lookup turns them into numbers. Blank units cannot be repaired, so we exclude those rows and count them. Then we compare with the true total. The two totals will differ, and the difference has a name.

Ask for the prediction first: Predict whether the cleaned revenue will be above, equal to or below the true revenue of 6375900.

```
words = {"one": 1, "two": 2, "three": 3, "four": 4, "five": 5}
work["units"] = pd.to_numeric(work["units"].replace(words), errors="coerce")
excluded = work[work["units"].isna()]
clean = work.dropna(subset=["units"]).copy()
clean["units"] = clean["units"].astype(int)
clean["line_value"] = clean["units"] * clean["unit_price_ugx"]
print(len(excluded), len(clean))
print(clean["line_value"].sum())
true_sales = pd.read_csv("sunrise_sales.csv")
true_total = (true_sales["units"] * true_sales["unit_price_ugx"]).sum()
residual = true_total - clean["line_value"].sum()
print(residual)
```

Correct result: 40 rows are excluded and 1473 rows are clean. The cleaned revenue is 6250500, which is 125400 below the true 6375900. The residual of 125400 is the value of the 40 excluded sales. It is not a mistake in the pipeline. It is the cost of rows that cannot be recovered.

Investigate, with answers:

- Why is the cleaned total below the true total? Answer: The 40 blank-unit sales are missing from the clean table, and their value is what is left over.
- Would it be honest to report that the two totals match? Answer: No. They differ by 125400, and the report says so and says why.

Modify: Prove that the residual equals the value of the 40 excluded sales, using the true sales table. Answer: Both are 125400.

```
excluded_value = (true_sales[true_sales["sale_id"].isin(excluded["sale_id"])]["units"] *
    true_sales[true_sales["sale_id"].isin(excluded["sale_id"])]["unit_price_ugx"]).sum()
```

## Make: reference

```
import pandas as pd
products = pd.read_csv("sunrise_products.csv")
lookup = {n.title(): n for n in products["name"]}
words = {"one": 1, "two": 2, "three": 3, "four": 4, "five": 5}

def clean_sales(raw):
    work = raw.drop_duplicates().copy()
    iso = pd.to_datetime(work["sale_date"], format="%Y-%m-%d", errors="coerce")
    slash = pd.to_datetime(work["sale_date"], format="%d/%m/%Y", errors="coerce")
    work["sale_date"] = iso.fillna(slash)
    work["product"] = work["product"].str.strip().str.title().map(lookup)
    work["units"] = pd.to_numeric(work["units"].replace(words), errors="coerce")
    excluded = work[work["units"].isna()]
    clean = work.dropna(subset=["units"]).copy()
    clean["units"] = clean["units"].astype(int)
    clean["line_value"] = clean["units"] * clean["unit_price_ugx"]
    return clean, excluded

clean, excluded = clean_sales(pd.read_csv("sunrise_sales_raw.csv"))
assert len(clean) == 1473
assert clean["line_value"].sum() == 6250500
print(len(excluded))
# The residual is 125400, the value of the 40 sales with blank units.
```

Build log entry to look for: DA16.ipynb with the function, two passing asserts, and a sentence naming the residual of 125400 and its cause.

## Misconceptions to listen for

- A clean file must match the true total exactly.
- Blank units can be replaced by the mean.
- Cleaning order does not matter.

## Questions to ask, with expected answers

- Why do the cleaned and true totals differ? Expected: The 40 blank-unit sales cannot be recovered, and their value is the residual.
- Why put the steps in a function? Expected: It reruns the same steps every time and can be tested with assert.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, Dates and names: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, Units, blanks and the residual: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (Dates and names): Count how many rows used the slash format.
2. Modify, Block B (Units, blanks and the residual): Prove that the residual equals the value of the 40 excluded sales, using the true sales table.
3. Make: Put the cleaning steps in one function called clean_sales(raw) that returns the clean table and the excluded rows. Run it on the raw export and check with assert that the clean table has 1473 rows and the clean revenue is 6250500. Write one sentence naming the residual and its cause.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: Why do the cleaned and true totals differ? Expected: The 40 blank-unit sales cannot be recovered, and their value is the residual.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA17: Merging two tables and the fan-out trap

Private. Keep this file out of the student folder.

**Objective.** By the end: you can merge two tables on a key, check the row count before and after, and spot a duplicated key that inflates a total.

**Question of the day.** Which product category earns the most, and can we trust the merge that tells us?

> **WHERE THIS IS USED**
> Customers, products and sales usually live in separate tables, and analysts join them. A careless join can count one sale twice. How would you check that your total is still right after a join?

Prerequisite: DA16 Cleaning the raw export.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

Where it overruns: The Block B surprise is the lesson. Let students find the extra 115 rows themselves before you explain.

## Block A. A merge you can trust

sales.csv holds the product_id of each sale. products.csv says which category each product belongs to. Merge them on product_id. The row count must not change, because each sale matches exactly one product.

Ask for the prediction first: Predict the row count after the merge and which category earns the most.

```
import pandas as pd
sales = pd.read_csv("sunrise_sales.csv")
products = pd.read_csv("sunrise_products.csv")
sales["line_value"] = sales["units"] * sales["unit_price_ugx"]
print(products["product_id"].is_unique)
merged = sales.merge(products[["product_id", "category"]], on="product_id", how="left")
print(len(sales), len(merged))
by_category = merged.groupby("category")["line_value"].sum().sort_values(ascending=False)
print(by_category)
print(by_category.sum())
```

Correct result: The key is unique (True). Both tables have 1513 rows. The categories rank Household 2082700, Drinks 1863200, Stationery 1686500 and Snacks 743500, and they add to 6375900. The count and the total both match the true values, so the merge is safe.

Investigate, with answers:

- Why check that product_id is unique before merging? Answer: If a product appeared twice, every sale of it would be copied twice.
- How do you know the merge did not lose or add rows? Answer: The row count is 1513 before and after, and the category totals add to the overall total.

Modify: Show the category with the lowest revenue. Answer: Snacks with 743500.

```
worst = by_category.idxmin()
```

## Block B. The fan-out trap

sunrise_products_practice.csv is a copy of the products table with one product listed twice. Merge with it and watch the totals. No error appears.

Ask for the prediction first: Predict what happens to the row count and the total revenue.

```
practice = pd.read_csv("sunrise_products_practice.csv")
print(practice["product_id"].is_unique)
bad = sales.merge(practice[["product_id", "category"]], on="product_id", how="left")
print(len(bad), bad["line_value"].sum())
print(bad["line_value"].sum() - sales["line_value"].sum())
print(practice[practice.duplicated("product_id", keep=False)])
```

Correct result: The key is not unique (False). The merged table has 1628 rows, which is 115 too many, and the total is 7079900, which is 704000 too high. The duplicate is Juice with product_id 6. Python raised no error. Only the check of the row count and total finds it.

Investigate, with answers:

- Why did Python not warn us? Answer: A merge with a repeated key is legal. It simply copies the matching rows.
- What two numbers give the problem away? Answer: The row count rose from 1513 to 1628, and the total rose from 6375900 to 7079900.

Modify: Ask the merge to check the key by adding validate="many_to_one". Read the result. Answer: pandas stops with a MergeError.

```
try:
    sales.merge(practice[["product_id", "category"]], on="product_id",
        validate="many_to_one")
    outcome = "no error"
except Exception as error:
    outcome = type(error).__name__
```

## Make: reference

```
import pandas as pd
sales = pd.read_csv("sunrise_sales.csv")
branches = pd.read_csv("sunrise_branches.csv")
sales["line_value"] = sales["units"] * sales["unit_price_ugx"]
named = sales.merge(branches[["branch_id", "branch_name"]], on="branch_id", how="left",
    validate="many_to_one")
assert len(named) == len(sales)
by_name = named.groupby("branch_name")["line_value"].sum()
print(by_name)
# Central Market 2997000, Campus Gate 1985400, Roadside 1393500. They match session 9.
```

Build log entry to look for: DA17.ipynb with the unchanged row count, three branch totals and the reconciliation sentence.

## Misconceptions to listen for

- A merge that runs without an error is correct.
- A merge never changes the row count.
- Any shared column can be a key.

## Questions to ask, with expected answers

- What two numbers tell you a merge went wrong? Expected: The row count and the grand total.
- What does validate="many_to_one" check? Expected: That the key is unique in the second table, so each row of the first matches one row.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, A merge you can trust: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, The fan-out trap: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (A merge you can trust): Show the category with the lowest revenue.
2. Modify, Block B (The fan-out trap): Ask the merge to check the key by adding validate="many_to_one". Read the result.
3. Make: Merge the sales with sunrise_branches.csv on branch_id to get the branch names. Check that the row count is unchanged. Total the revenue by branch name and reconcile it with the totals you

found in session 9.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: What two numbers tell you a merge went wrong? Expected: The row count and the grand total.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA18: Does your total match the true total? Clean, merge and reconcile

Private. Keep this file out of the student folder.

**Objective.** By the end: you can clean a raw export, merge it with a lookup table, and report a total together with its named difference from the true total.

**Question of the day.** Which category earns the most in the cleaned data, and by how much do our totals fall short of the truth?

> **WHERE THIS IS USED**
> Accountants and auditors compare two totals and explain the gap. Banks and tax teams do this every month. Can you explain why your total is a little short of the true one?

Prerequisite: DA17 Merging two tables and the fan-out trap.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 15 | recall, then read the brief together |
| 80 | learners work and the instructor visits each of the five |
| 15 | break away from screens |
| 40 | learners finish, restart and run all |
| 20 | a quiet one-to-one chat, then save and share |
| 10 | review |

Where it overruns: Use the error cards from sessions 10, 16 and 17 for repair. Introduce no new syntax.

## Make: reference

```
import pandas as pd
raw = pd.read_csv("sunrise_sales_raw.csv")
products = pd.read_csv("sunrise_products.csv")
lookup = {n.title(): n for n in products["name"]}
words = {"one": 1, "two": 2, "three": 3, "four": 4, "five": 5}
work = raw.drop_duplicates().copy()
work["sale_date"] = pd.to_datetime(work["sale_date"], format="%Y-%m-%d",
    errors="coerce").fillna(pd.to_datetime(work["sale_date"], format="%d/%m/%Y",
    errors="coerce"))
work["product"] = work["product"].str.strip().str.title().map(lookup)
work["units"] = pd.to_numeric(work["units"].replace(words), errors="coerce")
excluded = work[work["units"].isna()]
clean = work.dropna(subset=["units"]).copy()
clean["units"] = clean["units"].astype(int)
clean["line_value"] = clean["units"] * clean["unit_price_ugx"]
clean = clean.merge(products.rename(columns={"name": "product"})[["product",
    "category"]], on="product", how="left", validate="many_to_one")
assert len(clean) == 1473
by_category = clean.groupby("category")["line_value"].sum().sort_values(ascending=False)
true_sales = pd.read_csv("sunrise_sales.csv")
true_total = int((true_sales["units"] * true_sales["unit_price_ugx"]).sum())
residual = true_total - int(clean["line_value"].sum())
assert by_category.sum() == clean["line_value"].sum()
print(by_category)
print(residual)
# Household earns the most. The residual 125400 is the value of the 40 excluded rows.
```

Build log entry to look for: DA18.ipynb that reruns from a restarted kernel, with rows kept and excluded, the unique-key check, category totals, the named residual and two limitations. It goes in the build log, and the instructor gives one strength and one next step in a private chat.

## Misconceptions to listen for

- A passing merge means correct totals.
- The residual means the pipeline is wrong.
- More steps mean a better result.

## Questions to ask, with expected answers

- How do you prove the merge did not change the data? Expected: The row count and the grand total are the same before and after.
- What is the residual here? Expected: 125400 shillings, the value of the 40 sales with blank units.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Read the brief together on the shared screen, then the first build step. Each learner shares their screen in turn. |
| 5 | Break away from the screen |
| 35 | Learners keep building. The instructor visits each learner by screen share, about seven minutes each. |
| 10 | Close: say the exit sentence aloud, then the rest of the build task is set |

### After-class build task, about an hour

1. Make: In DA18.ipynb start from sunrise_sales_raw.csv. 1) Clean it with a function and report the rows kept, rows excluded and the cleaning steps. 2) Merge the clean table with the products table on product_id. Check that the key is unique and that the row count is unchanged. 3) Total the revenue by category. 4) Compare the cleaned total with the true total in sunrise_sales.csv. Name the residual and its cause. 5) Write two limitations. Run from a restarted kernel, save DA18.ipynb with a short explanation of your checks, and share them by the route in START_HERE.md.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: How do you prove the merge did not change the data? Expected: The row count and the grand total are the same before and after.

This is a work session. Keep the live part for the brief and for individual check-ins. The learner builds in the live time and after class.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA19: Project question and a safe plan

Private. Keep this file out of the student folder.

**Objective.** By the end: you can write a narrow project plan with a question, the data, the measures, the charts and the limits, and have it approved.

**Question of the day.** Which three questions can our data answer, and how will we show the answers?

> WHERE THIS IS USED
> Data teams write a short plan before any code: the question, the data, the charts and what the data cannot say. Which three questions about a topic you love could a table answer?

Prerequisite: DA18 Does your total match the true total? Clean, merge and reconcile.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

Where it overruns: Approve plans one at a time. A plan that is too big is the most common reason a project fails in week 8.

## Block A. From a topic to three questions

A project serves a reader. Start with the topic on your progress map. Decide who needs the answer and what choice it could help with. Review AI_USE_POLICY.md now. Any permitted supervised AI help is written in AI_USE_LOG.md, and your own attempt always comes first. No forecasting or model training is required.

Ask for the prediction first: Predict which of these questions the Sunrise Shops data can answer with the taught methods: (a) Which category earns the most at each branch? (b) Why do people like Campus Gate? (c) What will sales be next month?

Correct result: Question (a) can be answered: it needs the sales, products and branches tables, a merge and a group total. Question (b) asks for reasons, and the data has no opinions. Question (c) asks for a forecast, which this course does not teach. A narrow descriptive question with named columns is the right size.

Investigate, with answers:

- What makes question (a) answerable? Answer: Every column it needs exists, and the method is one we have used.

- How would you narrow question (b) so the data can answer it? Answer: Ask something countable, such as which shop has the highest revenue per customer.

Modify: Write one more question the Sunrise data can answer, and name the columns it needs. Answer: Any narrow question with named columns, for example: did the Sugar price rise change its units sold? It needs product_id, units and sale_date.

## Block B. The plan and the data

Your PROJECT_PLAN.md has seven headings: question, reader and use, data source and licence, measures and columns, method, charts, limitations. Start with Sunrise Shops, which is invented and needs no permission. You may use a public real dataset, for example from the Uganda Bureau of Statistics or the World Bank, once the licence has been read at the source, the instructor has approved it, and it holds no personal information.

Ask for the prediction first: Predict which heading is hardest to write, and why.

Correct result: Most learners find limitations hardest, because it means saying what the data cannot show. A good limitation names a missing column or a kind of conclusion the project will not draw: no costs means no profit, invented data means no claim about real shops.

Investigate, with answers:

- Why must the licence be read at the source? Answer: The licence says what you may do with the data. Only the owner of the data can tell you.
- What goes in the data dictionary? Answer: Each column you use, its meaning and its unit.

Modify: Write the limitations heading for your own plan in two sentences. Answer: Two sentences naming what the data cannot show.

## Make: reference

An acceptable plan names Sunrise Shops (invented) or an approved public dataset with its licence read at the source. Its three questions are descriptive and name their columns. Example: (1) Which category earns the most at each branch? Needs sales, products, branches. (2) Which branch earns the most per customer counted? Needs sales and daily_summary. (3) Did the Sugar price rise on day 31 change its units sold? Needs product_id, units, sale_date. Limitations: invented data, no costs so no profit, no claim about real shops. If the plan promises profit, change the measure to revenue. If it promises a forecast, make it descriptive.

Build log entry to look for: PROJECT_PLAN.md approved by the instructor, with a data dictionary for the columns used.

## Misconceptions to listen for

- A bigger question is a better project.
- Real data is always better than invented data.
- Limitations make the project look weak.

## Questions to ask, with expected answers

- Which columns does your first question need? Expected: Each one is named, and each is in the data dictionary.
- Who is the reader and what could they decide? Expected: A named reader and a concrete choice.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Block A, From a topic to three questions: predict, run, investigate. The Modify step moves to the build task. |
| 5 | Break away from the screen |
| 35 | Block B, The plan and the data: predict, run, investigate. The Modify step moves to the build task. |
| 10 | Close: say the exit sentence aloud, then the build task is set |

### After-class build task, about an hour

1. Modify, Block A (From a topic to three questions): Write one more question the Sunrise data can answer, and name the columns it needs.
2. Modify, Block B (The plan and the data): Write the limitations heading for your own plan in two sentences.
3. Make: Write PROJECT_PLAN.md with the seven headings. Start from the topic on your progress map and choose the supplied data file that fits it best. Real public data is used only after you and the instructor have read its licence. Name the data file and its licence or its invented status. List the three questions, the columns each needs, the charts and the limitations. Ask the instructor to approve a narrow scope before you build.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: Which columns does your first question need? Expected: Each one is named, and each is in the data dictionary.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA20: Build a reliable analysis

Private. Keep this file out of the student folder.

**Objective.** By the end: you can build the calculations for your project so that each one is checked by hand and by a second route, and a partner can rerun them.

**Question of the day.** Can a stranger follow my steps from the raw file to my numbers?

> WHERE THIS IS USED
> A result is only useful if someone else can repeat it. Analysts log each step and check each number two ways. Who could rerun your analysis and get the same numbers?

Prerequisite: DA19 Project question and a safe plan.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 15 | recall, then plan check |
| 50 | build calculations with hand checks |
| 15 | break away from screens |
| 50 | build second routes and reconcile |
| 35 | partner reruns one calculation and returns feedback |
| 15 | exit ticket, save and individual check |

Where it overruns: Each learner needs the instructor for a few minutes. Plan the order of visits before the session.

## Make: reference

Check that: the raw file is untouched; the cleaning log counts every row removed or excluded; each calculation has a hand check on a five-row sample; each total reconciles with a second route; row counts are printed after every merge; the partner can rerun the notebook from a restarted kernel.

Build log entry to look for: A notebook that reruns from the top, a cleaning log, and a partner note recording one rerun and the first discrepancy fixed.

## Misconceptions to listen for

- If the code runs, the numbers are right.
- A cleaning log is only needed when something was changed.
- A partner rerun is a test of the partner.

## Questions to ask, with expected answers

- How do you check a total you cannot count by hand? Expected: Take five rows, work them out on paper, and compare with the code on the same five rows.

- What does your row count show after the merge? Expected: It matches the count before the merge, or you can explain every extra row.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Read the brief together on the shared screen, then the first build step. Each learner shares their screen in turn. |
| 5 | Break away from the screen |
| 35 | Learners keep building. The instructor visits each learner by screen share, about seven minutes each. |
| 10 | Close: say the exit sentence aloud, then the rest of the build task is set |

### After-class build task, about an hour

1. Make: Work from a protected copy of the raw data. Write the cleaning log, including a log that says no change was needed if that is true. Build one calculation at a time. For each, compare with a small hand-worked example and with a second route (pandas against NumPy, or a merged total against an unmerged one). Ask a partner to rerun one calculation from your notebook. Write down the first discrepancy they find and fix it.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: How do you check a total you cannot count by hand? Expected: Take five rows, work them out on paper, and compare with the code on the same five rows.

This is a work session. Keep the live part for the brief and for individual check-ins. The learner builds in the live time and after class.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA21: Build and check the chart report

Private. Keep this file out of the student folder.

**Objective.** By the end: you can build a chart report that a stranger reads in 30 seconds, and you can critique a coding assistant's hint with a test.

**Question of the day.** Does my report answer my questions without a verbal tour?

> **WHERE THIS IS USED**
> A report that needs its author in the room has not done its job. Clear titles, labels and units let a farmer, a manager or a parent read it alone. Could your report speak for itself?

Prerequisite: DA20 Build a reliable analysis.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 10 | Recall: retrieve yesterday, three questions on paper |
| 50 | Block A: predict, run, investigate, modify |
| 15 | Break away from screens |
| 50 | Block B: predict, run, investigate, modify |
| 40 | Make: the independent task and the project journal |
| 15 | Exit ticket, save, and an individual check with each learner |

Where it overruns: The 20-minute critique lab sits inside the Make block. Start it only for learners who passed the individual readiness check.

## Make: reference

```
import pandas as pd
import matplotlib.pyplot as plt
sales = pd.read_csv("sunrise_sales.csv")
sales["line_value"] = sales["units"] * sales["unit_price_ugx"]
by_branch = sales.groupby("branch_id")["line_value"].sum()
plt.bar(by_branch.index.astype(str), by_branch.values)
plt.ylabel("Revenue (UGX)")
plt.title("Revenue by branch, Sunrise Shops (invented data)")
plt.ylim(bottom=0)
plt.savefig("revenue.png", bbox_inches="tight")
plt.show()
with open("revenue.png", "rb") as f:
    header = f.read(4)
# savefig writes the image and bbox_inches="tight" keeps the labels in.
```

Build log entry to look for: A chart report whose headline numbers match the notebook, a peer note on where they hesitated, and AI_USE_LOG.md with the tested critique.

## Misconceptions to listen for

- More charts make a stronger report.
- A plausible AI answer is a correct one.
- The report is allowed to need a spoken explanation.

## Questions to ask, with expected answers

- Where did your peer hesitate, and what did you change? Expected: A real hesitation and a concrete change to a heading or label.
- How did you test the assistant's advice? Expected: Predict, run, compare with the known answer, and keep the failing case.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Read the brief together on the shared screen, then the first build step. Each learner shares their screen in turn. |
| 5 | Break away from the screen |
| 35 | Learners keep building. The instructor visits each learner by screen share, about seven minutes each. |
| 10 | Close: say the exit sentence aloud, then the rest of the build task is set |

### After-class build task, about an hour

1. Make: Build the chart report. Use few charts, readable labels with units, the covered dates and a visible data source note. Write two findings and one limitation next to their evidence. Give a peer 30 seconds to find the largest category and its total, and record where they hesitate. To put a chart in your report, call plt.savefig("revenue.png", bbox_inches="tight") before plt.show(), then insert the PNG with Insert and Pictures. When the instructor confirms you are ready, spend 20 minutes with AI_CODING_ASSISTANTS_LAB.md: critique a hint on the practice function, run a test and log it in AI_USE_LOG.md.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: Where did your peer hesitate, and what did you change? Expected: A real hesitation and a concrete change to a heading or label.

This is a work session. Keep the live part for the brief and for individual check-ins. The learner builds in the live time and after class.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA22: Report and handover

Private. Keep this file out of the student folder.

**Objective.** By the end: you can hand over a project that another person can open, rerun and check.

**Question of the day.** Does everything I share open, rerun and agree with itself?

> WHERE THIS IS USED
> Before a team hands work over, they restart everything and run it from the top to prove it works. A tidy handover saves other people days. Does your notebook run from the first cell to the last?

Prerequisite: DA21 Build and check the chart report.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 15 | recall, then the handover checklist |
| 60 | rerun from a restarted kernel and repair |
| 15 | break away from screens |
| 50 | write and export the report |
| 25 | individual check and share |
| 15 | feedback, then save |

Where it overruns: Check each learner's folder arrived. A file on a laptop has not been shared.

## Make: reference

Checklist: the notebook reruns from a restarted kernel; the cleaning log counts each row removed; the data dictionary covers every column used; each finding in the report has a number in an output cell; the report states the invented or licensed data source and its limits; the log records any permitted AI help or says No AI assistance used; every file opens after being reopened.

Build log entry to look for: All five files saved together and reopened, and totals in the report that agree with the notebook.

## Misconceptions to listen for

- A polished picture is enough evidence.
- A saved file has been shared.
- Restarting the kernel is only for beginners.

## Questions to ask, with expected answers

- How do you know your report matches your notebook? Expected: Each number in the report is copied from an output cell that reruns from the top.

- What did you decide not to claim? Expected: A limitation named in the report, such as no costs so no profit.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Read the brief together on the shared screen, then the first build step. Each learner shares their screen in turn. |
| 5 | Break away from the screen |
| 35 | Learners keep building. The instructor visits each learner by screen share, about seven minutes each. |
| 10 | Close: say the exit sentence aloud, then the rest of the build task is set |

### After-class build task, about an hour

1. Make: Restart the kernel and run all cells from the top. Confirm every number in the report appears in an output. Write the one-page report: question, data, method, three findings with their values, one recommendation and one limitation. Export it to PDF and reopen it. Save project.ipynb, cleaning_log.md, your data dictionary, report.pdf or report.docx and AI_USE_LOG.md together in your project folder, and share the folder by the route in START_HERE.md. Keep personal information out of every file.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: How do you know your report matches your notebook? Expected: Each number in the report is copied from an output cell that reruns from the top.

This is a work session. Keep the live part for the brief and for individual check-ins. The learner builds in the live time and after class.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA23: Explain it to a parent

Private. Keep this file out of the student folder.

**Objective.** By the end: you can explain your project to a listener who has never seen code, in five minutes, with a chart and a limit.

**Question of the day.** Can a parent say what I found, how I know, and what the data cannot tell us?

> WHERE THIS IS USED
> Analysts explain their findings to busy people who do not write code. They say what they found, how they know, and what the data cannot tell. How would you say it to a parent in one minute?

Prerequisite: DA22 Report and handover.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 15 | recall, then the five-slide plan |
| 45 | build the slides |
| 15 | break away from screens |
| 60 | timed rehearsals in turn with peer feedback |
| 30 | repair, backup and the optional modern AI orientation |
| 15 | exit ticket, then save |

Where it overruns: Five learners at five minutes each plus questions fill the rehearsal block. Rotate the listener role.

## Make: reference

Five slides, one idea each, no code on screen unless it answers a question. A listener can say what was compared, the unit, the biggest difference and the limitation. The speaker can point to the notebook cell behind each number. A screenshot backup exists.

Build log entry to look for: showcase.pptx and a PDF backup, a timed rehearsal and a feedback note from a peer.

## Misconceptions to listen for

- A good talk lists the libraries used.
- More slides mean a stronger talk.
- A live demonstration must always run.

## Questions to ask, with expected answers

- What would a parent say your project found? Expected: A plain sentence with one number and one comparison.
- What is your backup if the laptop fails? Expected: A PDF and screenshots, and a spoken explanation.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 5 | Recall: three questions on paper, then say the answers aloud |
| 35 | Read the brief together on the shared screen, then the first build step. Each learner shares their screen in turn. |
| 5 | Break away from the screen |
| 35 | Learners keep building. The instructor visits each learner by screen share, about seven minutes each. |
| 10 | Close: say the exit sentence aloud, then the rest of the build task is set |

### After-class build task, about an hour

1. Make: Make five slides: my question; what data I used; what I did; my most useful finding with its chart; the limitation and next step. Save as showcase.pptx and keep a PDF and screenshots as a backup. Rehearse with a timer. Practise answering: where did this number come from, what did you clean, and what can this data not tell us. Disclose any help or AI use. Show only invented or approved non-personal data. The instructor may follow the rehearsal with MODERN_AI_ORIENTATION.md, a 20-minute optional introduction with no assignment.

When you finish, write two sentences: what you changed and how you checked it. Add a build log entry. Send a photo of your work or the notebook by the route the organiser sets, before the next class.

### Next class

In the next class recall, ask: What would a parent say your project found? Expected: A plain sentence with one number and one comparison.

This is a work session. Keep the live part for the brief and for individual check-ins. The learner builds in the live time and after class.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.

# DA24: Parent showcase and next steps

Private. Keep this file out of the student folder.

**Objective.** By the end: you can present your project, answer questions, and try a small change to it live with no AI assistant open.

**Question of the day.** What can I show and explain, and what will I learn next?

> WHERE THIS IS USED
> Start-ups and tech hubs hold demo days where builders show what they made and answer questions. Today is your demo day. What do you want your family to remember about your work?

Prerequisite: DA23 Explain it to a parent.

## The 180 minutes

| Minutes | Activity |
|---|---|
| 30 | setup, rehearsal and backups |
| 10 | welcome for parents and guardians |
| 40 | presentations and questions |
| 10 | break |
| 20 | feedback, certificates and photographs only where consent exists |
| 70 | reflection, project folder copy and next steps |

Where it overruns: A combined event for several groups needs a separate, longer slot than this plan allows.

## Make: reference

Use the SHOWCASE_GUIDE.md checklist as a guide for friendly feedback. Typical suggested changes: filter one branch, change a threshold from 5 units to 10, add a label to a chart, or rerun a calculation with a different column. The certificate goes to learners who attended or made up most classes, kept their build log and showed a finished project. The camp has no tests and no scores. The programme issues the certificate. No university or vendor qualification is involved.

Build log entry to look for: A showcase talk, one small live change, one strength and one next step, and a certificate decision based on attendance, build log and finished project.

## Misconceptions to listen for

- The showcase is a performance.
- Help must be hidden.
- A certificate certifies a profession.

## Questions to ask, with expected answers

- What did you change in the live task, and how did you check it? Expected: A concrete change and a check against a known value.
- What will you learn next? Expected: A specific skill with a first step.

## Online version

For a hybrid group. The live class is 90 minutes on video with screen sharing. The learner finishes the build in their own time, about an hour. The full rules are in handbook/HYBRID_DELIVERY.md.

The showcase is best held in person. Use this clock only when a family cannot come. No recording without written consent.

### Live clock, 90 minutes

| Minutes | Activity |
|---|---|
| 15 | Tech check, screen share test and the backup PDF ready on each laptop |
| 5 | Welcome for families on the video call |
| 45 | Presentations and questions: about nine minutes for each learner, five minutes of talk and the rest for questions |
| 15 | Feedback for every learner: one strength and one next step, then the next learning goal |
| 10 | Close: thanks, the project folder is sent by the route the organiser sets |

### After-class build task, about an hour

No build task. Send the project folder by the route the organiser sets, and keep a backup copy.

### Next class

There is no next class. Use the first minutes of any follow-up call to ask what each learner will learn next.

## Build log and feedback

Look at what each of the five learners saved and note one strength and one next step for each. Share them kindly and privately. Nothing is scored or ranked. Visit every learner, not only volunteers. Keep this guide private. Numeric results come from reference_values.json; explanations and show-and-tells need your judgement.